

NYYG · ARA SINAV

Nesneye Yonelik Yazilim Gelistirme

Ara Sinav Cevap Belgesi

Bilisim Enstitusu · 2025-2026 Bahar

Teslim Tarihi	30 Nisan 2026
Teslim Format	PDF
Toplam Puan	100 (2 × 50)
Kapsam	Soru 1: Akilli Ev Sistemi · Soru 2: SimpleCar E-Satis

State

Calisma Modu

Mediator

Izleme Motoru

Observer

Sensor → Motor → Kullanici

Strategy

Odeme Yontemi

CoR

Siparis Kontrolleri

Command

Siparis Isleme Adimlari

Bu belge, sinavda istenen iki senaryo icin tasarim oruntusu secimleri, gerekceler, sinif diyagramlari, sinif/iliski aciklamalari ve calisma akislarini icermektedir. Her iki diyagramda UML 2.x notasyonu ile cizilmis; goruntuleme/etkilesim katmani, alan modeli ve servis katmani olarak organize edilmistir.

SORU 1 · 50 PUAN

Akilli Ev Yonetim Sistemi

State + Mediator + Observer Oruntulerinin Birlikte Kullanimi

1.1 Senaryonun Ozeti

Bir akilli ev cozumunde *aydinlatma* ve *isitma* sistemleri ile *kapi* ve *hareket* sensorleri yer almaktadır. Tum bilesenler verilerini bir *Akilli Ev Izleme Motoru*'na aktarir; karsiligindaki talimatlarini yerine getirir. Aydinlatma ve isitma sistemleri *Guvenli* ve *Tasarruflu* olmak uzere iki calisma sekline sahiptir. Bu iki sekil *baglantilidir*: bir sistemde tasarruflu mod secilirse digeri de tasarruflu moda gecmek zorundadir; ayni durum guvenli mod icin de gecerlidir. Calisma sekillerine bagli olarak davranis farkli olur (orn. tasarruflu modda isitma 18°C hedefler; guvenli modda 40°C panel sicakligi sabit tutulur). Kullanici, sistemleri mobil veya web uygulamasini uzereinden yonetir; kapi ve hareket sensorlerine ait bildirimler etkin ise dogrudan kullanicinin uygulamasina iletilir.

1.2 Senaryodan Cikarilan Tasarim Sorulari

- Davranis bagimlilik:** Aydinlatma ve isitma sistemleri ic durumlarina (modlarına) gore *farkli davranir*. Guvenli/Tasarruflu modlar, davranisi *polimorfik* olarak degistirir; `if/else` ile cozulmesi acik-kapali (OCP) prensibini ihlal eder.
- Sistemler arasi senkronizasyon:** Aydinlatma ve isitma *birbirini bilmek zorunda* degil; ancak modlari *birlikte* degismelidir. Bu coklu-nesneli koordinasyon dogrudan referansla degil, *arabulucu* uzereinden gerceklestirilmelidir.
- Olay yayini:** Sensorler verilerini iletmeli, fakat kime ileteceklerini *bilmek zorunda olmamali*. Bu klasik bir *yayinci-abone* problemidir.
- Bildirim hedefi cogul:** Kullanicinin birden fazla istemcisi (mobil/web) olabilir. Bildirimler arayuze bagimli yazilmamalidir.

1.3 Tasarim Oruntusu Secimleri ve Gerekceleri

Oruntu	Uygulanan Yer	Gerekce
State (GoF · Behavioral)	<code>CalismaModu</code> arayuzu ile <code>GuvenliMod</code> ve <code>TasarrufluMod</code> ; <code>EvSistemi.aktifMod</code> alani uzereinden tasiniyor.	Sistemin davranisi <i>ic durumuna</i> gore degisiyor; <code>komutCalistir(...)</code> cagrildiginda farkli davranis sergilenir. Yeni bir mod (orn. <i>UykuModu</i>) eklendiginde <code>EvSistemi</code> kodu degismeden uzatilabilir (OCP).

Oruntu	Uygulanan Yer	Gerekce
Mediator (GoF · Behavioral)	AkilliEvIzlemeMotoru : tum sistemler ve sensorleri tanir; sistemleri birbirine baglamadan onlarin koordinasyonunu saglar.	"Bir mod degisirirse digeri de degisir" kurali, sistemler birbirini referans almadan <i>tek bir noktadan</i> uygulanir. Sistemler/sensorler arasi <i>cok-yonlu kosulluluk</i> arabulucuda toplanir; baglilik (coupling) dusurulur.
Observer (GoF · Behavioral)	(a) Sensor → SensorGozlemcisi arayuzu (motoru icerecek sekilde). (b) AkilliEvIzlemeMotoru → KullaniciBildirimAlici arayuzu (mobil/web).	Sensorler ve motor, <i>kim dinliyor</i> bilgisinden bagimsiz veri yayar. Kullanici uygulamasini icin yeni bir kanal (orn. e-posta) eklendiginde mevcut kod degismez; yalnızca yeni bir gozlemci eklenir.

Neden tek bir oruntu yeterli degil? State olmadan davranis kosullarla; Mediator olmadan baglantili modlar dogrudan referanslarla; Observer olmadan ise sensor/motor sikica baglanir. Uc oruntu birbirinin kaldiraci olarak *dusuk-bagli, yuksek-uyumlu* bir mimari uretir.

1.4 Sinif Diyagrami



Soru 1 sinif diyagrami

Sekil 1.1 · Akilli Ev Yonetim Sistemi — Sinif Diyagrami (UML 2.x). Modlar State; izleme motoru Mediator; sensor ve bildirim baglantilari Observer kullanir.

1.5 Sinif Aciklamalari

Sinif / Arayuz	Stereotip	Sorumluluk
Kullanici	«entity»	Ev sahibinin kimligini ve bilgi-erisim noktalarini (mobil/web) tasir.
KullaniciBildirimAlici	«interface»	Bildirim ve durum yansitma yetenegini soyutlar; istemci tipi degisse de motor degismez.
MobilUygulama · WebUygulamasini	«concrete»	KullaniciBildirimAlici arayuzunu uygular; kanal-ozel goruntuleme yapar.
DurumOzeti	«value object»	Sistem ve sensor durumunun anlik fotografi; degismez (immutable) tasinir.
AkilliEvIzlemeMotoru	«mediator» · «observer»	Sistemleri ve sensorleri kompoze eder; mod senkronizasyonu, komut yonlendirme, sensor olaylarini alma ve bildirim yayini gorevlerini ustlenir.

Sinif / Arayuz	Stereotip	Sorumluluk
EvSistemi	«abstract»	Mod davranisini tutan ortak ust sinif; komutCalistir alt siniflara birakilir.
AydinlatmaSistemi · IsitmaSistemi	«concrete»	Cihaza ozel davranisi (asgari/maksimum aydinlatma, hedef sicaklik) uygular.
CalismaModu	«interface»	Ic durumun davranisini soyutlar (State); uygula(sistem) ile sistemi ayarlar.
GuvenliMod · TasarrufluMod	«concrete»	lki politik davranis: maksimum kapasite vs. asgari/hedef ayarlar.
ModTipi	«enumeration»	Kullanici arayuzunden gelen mod tercihi icin tip-guvenli sabitler.
Sensor	«abstract»	Subject rolu: attach/detach ve bildir ortak davranisini saglar.
KapiSensoru · HareketSensoru	«concrete»	Donaniml soyutlamalar; olcumGonder implementasyonlari.
SensorGozlemcisi	«interface»	Sensor olaylarini alacak nesnelere uygulanan Observer arayuzu.
SensorOlayi	«value object»	Tip, kaynak sensor, veri yuku ve zamani tasiyan degismez olay nesnesi.
SensorTipi	«enumeration»	Sensorun anlamsal sinifi (KAPI/HAREKET).

1.6 Tipik Calisma Akislari

Akis A · Kullanicinin tasarruflu mod talebi

- MobilUygulama üzerinden tumModlariAyarla(TASARRUFLU) cagrilir.
- AkilliEvIzlemeMotoru kayitli her EvSistemi icin modAyarla(new TasarrufluMod()) cagilir.
- Her sistem (aydinlatma ve isitma) *ayni anda* tasarruflu davranisa gecer; sistemler birbirini bilmez, fakat senkronize olur.
- Motor durumOzetiYayinla() ile guncel DurumOzeti 'ni tum bildirim alicilarina yansitir.

Akis B · Kapi sensorunden bildirim

- KapiSensoru.olcumGonder() cagrilir; bildir(new SensorOlayi(KAPI, ...)) tetiklenir.
- Sensor, kayitli tum SensorGozlemcisi nesnelere olayi iletir; AkilliEvIzlemeMotoru bunlardan biridir.

3. Motor, sensor etkin ise olayi `KullaniciBildirimAlici` uygulamalarina `bildirimGoster(olay)` ile yonlendirir.
4. Mobil ve web istemciler bildirimi kullaniciya kanal-ozel bicimde sunar.

1.7 Tasarim Avantajlari

- **Acik-Kapali (OCP):** Yeni bir mod (`UykuModu`), yeni bir sensor (`Pencere`), yeni bir bildirim kanali (E-posta) eklerken mevcut kod degismez.
- **Tek Sorumluluk (SRP):** Mod davranisi `CalismaModu` hiyerarsisinde; senkronizasyon politikasi `AkilliEvIzlemeMotoru` 'nda; olay tasimasi `Sensor` + `SensorGozlemcisi` ikilisinde.
- **Dusuk Baglilik:** Sistemler birbirini referans almaz; sensorler kullanici uygulamasini bilmez.
- **Test Edilebilirlik:** Stub `SensorGozlemcisi` ya da sahte `CalismaModu` ile birim test yapılabilir.

SORU 2 · 50 PUAN

SimpleCar E-Satis Sistemi

Strategy + Chain of Responsibility + Command Oruntulerinin Birlikte Kullanimi

2.1 Senaryonun Ozeti

Tek model araba ureten ve satan SimpleCar firmasi yalnızca e-satis yapmaktadır. Siparislerde kullanılan ödeme yöntemleri banka havalesi, paypal veya soguk cuzdan olabilir; bu yöntemler zaman icinde siklikla genisler veya daralir. Her siparis icin sahtecilik, limit ve bakiye kontrolleri yapılmalıdır; bu kontroller de zamanla artırilabilir. Herhangi bir kontrolun olumsuz sonuclanmasi siparisi iptal eder. Tum kontroller basariliysa siparis sirasiyla fatura duzenleme, fatura gonderme ve alacaklar listesine kaydetme adimlariyla islenir.

2.2 Senaryodan Cikarilan Tasarim Sorulari

- Algoritma ailesi (odeme):** Odeme tek bir davranistir, fakat birden fazla farkli yolla yapilir; ayrıca yöntemler siklikla genisler. Calisma zamaninda secilebilen ve genisletilebilen bir algoritma ailesi gerekir.
- Asamali ve genisleyebilir kontrol:** Kontroller artırilabilir; biri olumsuz olursa zincir kesilmeli ve siparis iptal edilmelidir. Bu, bir sorumluluk zinciri probleminin tipik formudur.
- Sirali tamamlanmasi gereken islem adimlari:** Fatura duzenleme → fatura gonderme → alacak kaydi adimlari sirayla yapılmalıdır; her adim baslibasina kapsullenmis ve test edilebilir bir ilave isi temsil eder. Adim sayisi ileride degisebilir.

2.3 Tasarim Oruntusu Secimleri ve Gerekceleri

Oruntu	Uygulanan Yer	Gerekce
Strategy (GoF · Behavioral)	OdemeYontemi arayuzu; BankaHavalesi, Paypal, SogukCuzdan.	Odeme algoritmasi siparisten bagimsiz olarak yer degistirebilmeli; yeni yontem eklemek mevcut kodu degistirmemeli.
Chain of Responsibility (GoF · Behavioral)	SiparisKontrolu soyut sinifi; SahtecilikKontrolu, LimitKontrolu, BakiyeKontrolu halkalari.	Her halka basariliysa siradakine devreder; basarisizsa zinciri keser ve siparis iptal edilir. Yeni bir kontrol eklemek tek satir konfigurasyondur (setSonraki).
Command + Macro Command	Komut arayuzu; FaturaDuzenle, FaturaGonder,	Her adim kapsullenmis bir nesne; sira degisikligi, ek adim, geri-alma

Oruntu	Uygulanan Yer	Gerekce
(GoF · Behavioral)	AlacaklaraKaydet ; SiparisIsleyici sirayla calistirir.	(undo) gibi ihtiyaclar icin esnek bir altyapi.

Tipik bir hatali tasarim: Odeme yontemini `Siparis` icine `if/switch` ; kontrolleri tek bir `kontrolEt()` metodunda `if` ; islem adimlarini sıra ile metoda gomerek yazmak, her yeni yontem/kontrol/adim eklemede *mevcut kodun* degismesine yol acar. Acik-Kapali ve Tek Sorumluluk prensipleri ihlal edilir. Yukaridaki uclu, bu hatayi yapisal olarak engeller.

2.4 Sinif Diyagrami



Soru 2 sinif diyagrami

Sekil 2.1 · SimpleCar E-Satis Sistemi — Sinif Diyagrami (UML 2.x). Odeme yontemleri Strategy; kontroller CoR; siparis isleme adimlari Command (Macro) ile organize edilir.

2.5 Sinif Aciklamalari

Sinif / Arayuz	Stereotip	Sorumluluk
Musteri	«entity»	Siparisi veren kisi; bakiye/limit hesaplamalari icin temel bilgi kaynagi.
Siparis	«entity»	Toplam tutar, miktar, secilen odeme yontemi ve durum bilgisini tasiyan toplama (aggregate root).
SiparisDurumu	«enumeration»	Siparis yasam dongusunu modelleyen sabitler.
OdemeYontemi	«interface»	Strategy: <code>odemeYap(tutar)</code> sozlesmesi.
BankaHavalesi · Paypal · SogukCuzdan	«concrete»	Mevcut odeme stratejileri; yenisi rahatca eklenir.
SiparisKontrolu	«abstract»	CoR taban sinifi; <code>kontrolEt</code> sablon metodu zincirleme isini ust sinifta tasir.
SahtecilikKontrolu · LimitKontrolu · BakiyeKontrolu	«concrete»	Tek bir kurali uygulayan kontrol halkalari; bagimsiz birim test yapılabilir.
Komut	«interface»	Command: <code>calistir(siparis)</code> sozlesmesi.
FaturaDuzenle · FaturaGonder · AlacaklaraKaydet	«concrete»	Senaryoda istenen sirali uc isleme adimi.

Sinif / Arayuz	Stereotip	Sorumluluk
SiparisIsleyici	«macro command»	Komut listesini <i>sirayla</i> calistirir; ek adim eklemek icin <code>komutEkle</code> .
SiparisYoneticisi	«service»	Use-case orchestratoru: kontrol zincirini koridor uzerinde calistirir; basarili ise isleyiciyi devreye alir, basarisiz ise siparisi iptal eder.
Fatura	«entity»	Fatura komutlari arasinda paylasilan veri; siparis ile 1-1 iliskili.

2.6 Siparis Yasam Dongusu — Akis Senaryosu

1. Musteri sepetini onaylar; sistem yeni bir `Siparis(durum=ALINDI)` olusturur ve secilen `OdemeYontemi` 'ni atar.
2. `SiparisYoneticisi.siparisAl(s)` cagrilir; siparis `KONTROL_EDILIYOR` durumuna gecer.
3. `kontrolZinciri.kontrolEt(s)` calisir:
 - `SahtecilikKontrolu` → basarili ise siradakine,
 - `LimitKontrolu` → basarili ise siradakine,
 - `BakiyeKontrolu` → sonuclanir.

Herhangi biri `false` donerse zincir kesilir, `siparis.iptalEt()` cagrilir ve durum `IPTAL_EDILDI` olur.

4. Tum kontroller basarili ise `siparis.odemeAl()` ile `OdemeYontemi.odemeYap(tutar)` calistirilir.
5. `SiparisIsleyici.tumKomutlariCalistir(s)` sirayla:
 - a. `FaturaDuzenle` → `Fatura` nesnesi olusturur,
 - b. `FaturaGonder` → faturayi musteriye gonderir,
 - c. `AlacaklaraKaydet` → alacaklar defterine kayit acar.
6. Siparis `ISLENDI` durumuna alinir; yonetici sonucu doner.

2.7 Genisletilebilirlik Senaryolari

Yeni Gereksinim	Mevcut Tasarimda Yapilacak Tek Sey
Kripto kart yontemi eklemek	<code>OdemeYontemi</code> 'ni uygulayan yeni bir sinif yazmak.
KYC kontrolu eklemek	<code>SiparisKontrolu</code> 'ndan turetilen yeni bir halka yazip zincire <code>setSonraki</code> ile takmak.

Yeni Gereksinim	Mevcut Tasarimda Yapilacak Tek Sey
Kargo etiketi olusturma adimi eklemek	Komut 'u uygulayan yeni bir sinifi SiparisIsleyici.komutEkle ile dahil etmek.
Sirali calistirma yerine bazi adimlari paralel calistirma	SiparisIsleyici 'yi ParalelSiparisIsleyici olarak alternatiflestirmek (Liskov uyumlu).

2.8 Sonuc

Onerilen tasarim; siparisin *alinmasi*, *dogrulanmasi* ve *islenmesi* ic akislarini birbirinden ayri, oruntu-tabanli kapsullere yerlestirir. Boylece sik degisen iki gereksinim — *odeme yontemleri* ve *kontrol kurallari* — mevcut kodu degistirmeden buyuyebilir; sirali isleme ise net, gozlemlenebilir ve test edilebilir bir komut akisi ile gerceklestirilir.